Keeping the dependency versions of all relevant libraries (like `@ai-sdk/*`, `ai`, and Next.js) up to date is an additional step in ensuring the reliability and performance of your application. Make sure to regularly update dependencies to take advantage of new features, performance improvements, and security patches.

8.2.2 Managing token limits and rate limiting

When working with AI providers, sooner or later, you will encounter problems; managing token limits and implementing rate limiting are key aspects to check so that your application runs smoothly and efficiently. If you run out of quota limits or you don't control the flow of requests, your services might get bogged down and stop working efficiently.

Here is a quick overview of some particular problems:

- *Token limits*—Token limits refer to the maximum number of tokens that can be processed within a given timeframe. Each AI provider has specific limits on the number of tokens that can be sent or received in requests, which can vary based on the pricing tier or plan. For example, Azure OpenAI API has defined limits such as requests per minute, requests per day, tokens per minute, and tokens per day. Exceeding these limits can result in errors or throttled responses.
- *Rate limiting*—Rate limiting is a technique used to control the number of incoming requests to an API within a specified timeframe. Each provider has its own rate limits imposed in its public-facing API. In addition, when exposing API endpoints in your own apps, adding rate limiting helps prevent abuse and ensures fair usage among all users.
- *Model fallbacks*—Model fallbacks are mechanisms that allow applications to switch between different AI models or providers when one becomes unavailable or encounters problems. This is to ensure that users will be served by an available service provider.

Since both token/rate limits and fallbacks have different meanings and recovery mechanisms compared to traditional error handling, it's important to create specific abstractions that manage both cases in your web application.

MANAGING TOKEN LIMITS

Token management should be a proactive process in general since we know that sent tokens that exceed the specified AI provider limits will be automatically rejected. We have a variety of strategies to prevent reaching those limits:

- *Use of* `maxTokens` *parameter*—The `maxTokens` parameter allows you to specify the maximum number of tokens for each response. This helps ensure that the generated output does not exceed the allowed token count according to the provider's constraints:

```
const response = await streamText({
  model: supportedModel,
```